

# Project Master Knowledge Guide

## Automated IT Support Ticket Classification and Routing System Using Natural Language Processing

This is the developer's complete learning guide for the implemented project. It is intended for NTCC viva, demonstration, faculty evaluation, project defense, and technical interviews. The repository is the source of truth. Whenever this guide says that something is implemented, the corresponding code or generated artifact exists in the repository.

Important truth boundary:

- The implemented and evaluated classifiers are TF-IDF + Logistic Regression, TF-IDF + Multinomial Naive Bayes, and DistilBERT.
- No LSTM model or Word2Vec implementation exists.
- Entity extraction is configurable and rule-based using spaCy `EntityRuler`; it is not a trained statistical NER model.
- Routing uses the predicted category. Entities and priority are returned and persisted, but they do not currently alter the assigned team.
- Docker configuration exists, but successful Docker runtime execution is **UNVERIFIED - REQUIRES REPOSITORY VALIDATION**.

## Repository Map

| Path                                           | Responsibility                                                |
|------------------------------------------------|---------------------------------------------------------------|
| data/<br>all_tickets_processed_improved_v3.csv | Source dataset                                                |
| data/ticket_routing.db                         | SQLite prediction database                                    |
| src/preprocessing/                             | Ticket-text normalization                                     |
| src/classification/                            | Baseline and DistilBERT training utilities                    |
| src/ner/                                       | Rule-based entity extraction                                  |
| src/priority/                                  | Rule-based priority assignment                                |
| src/routing/                                   | Category-to-team routing                                      |
| src/api/                                       | FastAPI application, schemas, dependencies, and service layer |
| src/database/                                  | SQLAlchemy model, engine, and repository                      |
| dashboard/                                     | Streamlit UI, components, charts, and API utilities           |
| scripts/                                       | EDA, training, and NER demonstration entry points             |
| models/                                        | Saved trained model artifacts                                 |

| Path                           | Responsibility                                              |
|--------------------------------|-------------------------------------------------------------|
| reports/                       | Generated EDA, model, design, validation, and audit reports |
| tests/                         | Unit and integration tests                                  |
| config/entity_patterns.json    | Configurable NER patterns                                   |
| Dockerfile, docker-compose.yml | Container deployment configuration                          |

---

## SECTION 1: PROJECT OVERVIEW

### 1.1 Problem Solved

IT service desks receive large numbers of free-text support tickets. A user may write:

"I am unable to log in to Outlook on my laptop."

A human analyst traditionally reads the ticket, decides its category, estimates urgency, identifies important technical details, and forwards it to a support team. This manual process is repetitive and vulnerable to inconsistency:

- Two analysts may categorize the same ticket differently.
- Tickets may wait in a general queue before being routed.
- Important details such as software names or error codes may be overlooked.
- High-impact incidents may not be recognized immediately.
- Manual routing becomes difficult as ticket volume grows.

This project automates a major portion of that workflow. It classifies each ticket into one of eight categories, extracts configured entities, assigns priority using deterministic business rules, maps the predicted category to a support team, persists the completed analysis, and returns the result through an API and dashboard.

### 1.2 Why NLP Is Suitable

The main input is natural-language text, not a fixed form. Users describe the same problem in many ways:

- "Cannot login"
- "Unable to log in"
- "Login failed"
- "I cannot access my account"

Natural Language Processing is suitable because it converts human-written text into information that software can use. In this project:

- TF-IDF converts words and phrases into sparse numeric features.
- Logistic Regression and Naive Bayes provide interpretable baselines.
- DistilBERT models contextual meaning and word relationships.
- spaCy EntityRuler extracts known technical phrases and strict error-code patterns.
- Keyword rules detect operational urgency.

## 1.3 Business Value

The system can reduce the time between ticket submission and assignment. Its business benefits include:

- Faster initial triage.
- More consistent category assignment.
- Reduced Level-1 service-desk workload.
- Immediate identification of high-priority phrases.
- Automatic routing to specialized teams.
- Persisted history for operational analytics.
- A repeatable API that can later integrate with an ITSM platform.

The system does not replace support engineers. It automates triage and routing so engineers can focus on diagnosis and resolution.

## 1.4 Technical Value

The project demonstrates an end-to-end applied NLP system rather than only a notebook model:

1. Real dataset analysis.
2. Reusable preprocessing.
3. Baseline model comparison.
4. Transformer fine-tuning.
5. Entity extraction.
6. Rule-based business logic.
7. REST API.
8. Database persistence.
9. Interactive dashboard.
10. Container configuration.
11. Automated tests.

It also demonstrates separation of concerns. Classification, entity extraction, priority assignment, routing, persistence, API delivery, and UI rendering are separate modules.

## 1.5 Real-World Use Cases

- **Employee access issue:** "Unable to log in to the employee portal." The system can classify it, assign High priority through the `cannot login` rule, and route the predicted category.
- **Hardware request:** "Need a second monitor for the new employee." The classifier can identify Hardware and route it to Hardware Team.
- **Storage request:** "Please increase shared mailbox storage." The classifier can predict Storage, the entity extractor can identify `mailbox` and `storage`, and routing sends it to Storage Team.
- **Production outage:** "Production server is down." The priority engine assigns Critical before lower-priority rules are considered.
- **Purchase request:** "Please purchase a headset." The ticket can be routed to Procurement Team.
- **Administrative rights request:** "Need admin rights for Visual Studio." The classifier can predict Administrative rights, extract Visual Studio and admin rights, and route to System Administration Team.

## 1.6 Benefits

- Modular, testable architecture.
- Real saved models and reproducible metrics.
- Contextual DistilBERT classification.
- Explainable deterministic routing and priority rules.
- Configurable NER patterns without retraining.
- Automatic OpenAPI documentation.
- Lightweight SQLite persistence.
- Accessible Streamlit demonstration interface.

## 1.7 Limitations

- Classification is single-label only.
- DistilBERT can still misclassify ambiguous tickets.
- NER recognizes configured patterns; unknown product names may be missed.
- NER has no labeled evaluation dataset, so precision/recall/F1 are unverified.
- Priority is keyword-based and can miss urgency expressed without configured terms.
- Routing depends only on the predicted category.
- No authentication, authorization, rate limiting, or user feedback mechanism is implemented.
- Dashboard priority and routing distribution charts use sample data rather than live

/analytics data.

- No LSTM model exists.
  - Production latency and scalability are unverified.
- 

## SECTION 2: END-TO-END SYSTEM FLOW

### 2.1 Complete Workflow

Ticket Input

```
-> Optional training-time preprocessing / runtime DistilBERT tokenization
-> DistilBERT classification
-> Entity extraction
-> Priority assignment
-> Category routing
-> SQLite persistence
-> FastAPI response
-> Streamlit display
```

The primary integrated entry point is `POST /analyze`.

### 2.2 Step 1: Ticket Input

**Input:** a JSON object containing a non-empty `ticket_text` string.

```
{"ticket_text": "Unable to login to Outlook on my laptop"}
```

**Code involved:**

- `src/api/schemas.py: PredictRequest`
- `src/api/app.py: analyze()`
- `dashboard/app.py: ticket text area and API calls`

Pydantic validates that `ticket_text` exists and has at least one character. Missing or empty values fail validation before business logic runs.

### 2.3 Step 2: Preprocessing and Tokenization

There are two important contexts:

1. **Training and EDA normalization:** `src/preprocessing/text.py` cleans text before baseline training and EDA.
2. **Production DistilBERT inference:** `TicketAnalysisService.predict()` directly passes ticket text to the saved DistilBERT tokenizer, which truncates and pads as required.

The DistilBERT tokenizer creates tensors such as token IDs and attention masks. Maximum inference length is 128 tokens.

## 2.4 Step 3: DistilBERT Classification

**Input:** raw ticket text.

**Internal processing:**

1. `DistilBertTokenizerFast` converts text into tensors.
2. Tensors move to CUDA if available, otherwise CPU.
3. `torch.no_grad()` disables gradient calculation for inference.
4. The model produces logits, one score for each of eight classes.
5. Softmax converts logits into probabilities.
6. The highest probability determines the label and confidence.

**Output:**

```
Prediction(category="Access", confidence=0.92)
```

**Code involved:** `src/api/services.py`,  
`models/distilbert/label_mapping.json`.

## 2.5 Step 4: Entity Extraction

**Input:** original ticket text.

**Internal processing:**

- `EntityExtractor` runs a blank English spaCy pipeline containing an `EntityRuler`.
- Patterns come from `config/entity_patterns.json`.
- Matches are returned with text, label, character offsets, and pattern ID.
- `TicketAnalysisService.extract_entities()` groups matches by label and removes duplicate values per label.

**Possible output:**

```
{  
  "SOFTWARE": ["Outlook"],  
  "DEVICE": ["laptop"]  
}
```

No matched entity means the grouped dictionary simply lacks that label. The API may return `{}` when nothing matches.

## 2.6 Step 5: Priority Assignment

**Input:** original ticket text.

**Internal processing:**

1. Convert text to lowercase.
2. Replace non-alphanumeric runs with spaces.

3. Evaluate priority groups in order: Critical, High, Medium, Low.
4. For each canonical keyword, also evaluate configured aliases.
5. Return the first highest-priority match.
6. If nothing matches, return Medium with `matched_rule=None`.

**Example output:**

```
{"priority": "High", "matched_rule": "cannot login"}
```

**Code involved:** `src/priority/rules.py`, `src/priority/engine.py`.

## 2.7 Step 6: Routing

**Input:** the predicted category, not the raw text.

**Internal processing:**

1. Validate category is a non-empty string.
2. Look up category in `CATEGORY_TO_TEAM`.
3. Return a frozen `RouteResult`.
4. Raise `UnsupportedCategoryError` if no mapping exists.

**Example output:**

```
{"category": "Access", "assigned_team": "Access Management Team"}
```

**Code involved:** `src/routing/rules.py`, `src/routing/router.py`.

## 2.8 Step 7: Database Persistence

Only the full `/analyze` workflow automatically persists a record.

**Stored values:**

- Original text.
- Predicted category.
- Confidence.
- Entities serialized as JSON text.
- Priority.
- Assigned team.
- UTC timestamp.

**Code involved:** `src/database/models.py`, `src/database/database.py`, `src/database/repository.py`, `src/api/services.py`.

## 2.9 Step 8: FastAPI Response

FullPipelineResponse validates the final response:

```
{
  "category": "Access",
  "confidence": 0.92,
  "entities": {"SOFTWARE": ["Outlook"], "DEVICE": ["laptop"]},
  "priority": "High",
  "matched_rule": "cannot login",
  "assigned_team": "Access Management Team"
}
```

## 2.10 Step 9: Streamlit Display

The dashboard calls `/analyze`, `/predict`, `/entities`, and `/priority`. It renders:

- Prediction and confidence.
- Entity groups.
- Priority and matched rule.
- Assigned team.
- Full analysis.

The repeated calls allow separate panels and a full-response panel, but they also mean one button click makes four backend requests. A future improvement would render everything from `/analyze` alone.

---

## SECTION 3: DATASET DEEP DIVE

### 3.1 Dataset Structure

**Path:** `data/all_tickets_processed_improved_v3.csv`

| Column      | Meaning                                            |
|-------------|----------------------------------------------------|
| Document    | Free-text IT support ticket                        |
| Topic_group | Ground-truth category used for classifier training |

Verified dataset facts:

- Rows: 47,837
- Columns: 2
- Missing documents: 0
- Missing labels: 0
- Duplicate rows: 0

- Classes: 8
- Largest class: Hardware, 13,617
- Smallest class: Administrative rights, 1,760
- Imbalance ratio: 7.7369

## 3.2 Class Distribution

| Category              | Count  | Percentage |
|-----------------------|--------|------------|
| Hardware              | 13,617 | 28.47%     |
| HR Support            | 10,915 | 22.82%     |
| Access                | 7,125  | 14.89%     |
| Miscellaneous         | 7,060  | 14.76%     |
| Storage               | 2,777  | 5.81%      |
| Purchase              | 2,464  | 5.15%      |
| Internal Project      | 2,119  | 4.43%      |
| Administrative rights | 1,760  | 3.68%      |

This imbalance matters because a model could appear accurate by favoring large classes. That is why the project reports macro F1 and uses stratified splitting. Logistic Regression also uses balanced class weights.

## 3.3 EDA Findings

- Average raw character length: 291.88
- Median raw character length: 175
- Average cleaned character length: 291.87
- Median cleaned character length: 175
- Average cleaned word count: 43.60
- Median word count: 26
- 90th percentile: 91 words
- 95th percentile: 136 words
- 99th percentile: 284 words
- Very short tickets under 3 words: 14

The 128-token DistilBERT maximum is a practical balance. It covers most ordinary tickets while limiting GPU memory and training time. Some long tickets are truncated.

## 3.4 Category Meanings and Examples

### Hardware

Physical equipment, endpoints, peripherals, or hardware incidents.

Examples:

- "Laptop keyboard is not working."
- "Need an additional monitor."
- "Printer is offline."

Why: the primary subject is physical equipment. Routed to Hardware Team.

### HR Support

Employee lifecycle, HR-related systems, allocation, and administrative support associated with HR operations.

Examples:

- "New starter setup required."
- "Payroll issue for employee."
- "Update employee allocation."

Why: the business owner is HR. Routed to HR Team.

### Access

Login, account access, permissions to services, password, VPN, or access-card requests.

Examples:

- "Unable to login to Confluence."
- "Need VPN access."
- "Please reset my password."

Why: the user is requesting or failing to use access. Routed to Access Management Team.

### Miscellaneous

Tickets that do not fit a more specific category or represent broad service-desk requests.

Examples:

- "Please restart the service."
- "General inquiry about support."
- "Expense report not found."

Why: no more specific supported category is dominant. Routed to Service Desk Team.

## Storage

Mailbox size, folders, shared drives, storage capacity, or file-space requests.

Examples:

- "Increase mailbox size."
- "Create shared mailbox."
- "Need more shared-drive storage."

Why: the core resource is storage. Routed to Storage Team.

## Purchase

Procurement requests for equipment, subscriptions, or other purchasable items.

Examples:

- "Please purchase a headset."
- "New purchase order required."
- "Need approval to buy a monitor."

Why: the action is procurement. Routed to Procurement Team.

## Internal Project

Project-code, internal project setup, or internal project administration.

Examples:

- "Create a new project code."
- "Delete the old internal project."
- "Set up project billing code."

Why: the ticket concerns internal project operations. Routed to Internal Projects Team.

## Administrative rights

Elevated permissions, administrator privileges, or software actions requiring administrative authority.

Examples:

- "Need admin rights for Visual Studio."
- "Administrator permission required to install software."
- "Windows upgrade needs elevated rights."

Why: the user needs privileged system access. Routed to System Administration Team.

## 3.5 Dataset Split

The shared split function in `src/classification/baselines.py` creates:

- Training: 38,269 rows, 80%
- Validation: 4,784 rows, 10%
- Test: 4,784 rows, 10%

It first separates 80% training data, then splits the remaining 20% equally. `stratify=y` preserves class proportions in every split. `random_state=42` makes the split reproducible.

---

## SECTION 4: PREPROCESSING PIPELINE

### 4.1 Code Location

- `src/preprocessing/text.py`
- Used by `scripts/run_phase1_eda.py`
- Used by baseline dataset loading in `src/classification/baselines.py`

### 4.2 Unicode Normalization

**Operation:** `unicodedata.normalize("NFKC", text)`

**Why:** visually similar Unicode characters may have different internal encodings. NFKC converts compatibility characters into consistent forms.

**Example:**

```
Input:  "Windows login"
Output: "Windows login" before lowercasing
```

Without it, equivalent words can become different features.

### 4.3 Lowercasing

**Operation:** `normalized.lower()`

**Why:** prevents "Outlook", "OUTLOOK", and "outlook" from becoming separate TF-IDF features.

```
Input:  "OUTLOOK Not Working"
Output: "outlook not working"
```

### 4.4 URL Removal

**Pattern:** `https?:\/\/\S+|www\.\S+`

**Why:** URLs are often unique noise and can massively expand vocabulary.

```
Input:  "Install VPN from https://example.com/download"
Output: "install vpn from"
```

URLs are removed, not replaced with placeholder tokens.

## 4.5 Email Removal

**Pattern:** a regular expression matching common email formats.

**Why:** email addresses are often personally identifying and usually do not help category prediction.

```
Input: "Contact admin@example.com for access"
Output: "contact for access"
```

## 4.6 Control-Character Normalization

Tabs, carriage returns, and newlines are replaced with spaces.

```
Input: "Need access\nUrgent\tplease"
Output: "need access urgent please"
```

This prevents formatting differences from affecting features.

## 4.7 Character Filtering

The filter removes characters outside:

```
a-z, 0-9, whitespace, period, underscore, colon, slash, backslash, hyphen
```

It deliberately preserves useful technical strings:

```
Windows 11 error 0x80070005 on C:\Temp\app.exe
```

The cleaned output still contains 0x80070005 and c:\temp\app.exe.

## 4.8 Standalone Punctuation and Whitespace Cleanup

Standalone technical punctuation is removed when it acts only as noise, and repeated whitespace collapses to one space.

```
Input: "Need access / urgently"
Output: "need access urgently"
```

## 4.9 Missing Values

If `pandas.isna(text)` is true, `clean_ticket_text` returns an empty string. Dataset loading later removes empty cleaned text before training.

## 4.10 What Is Not Implemented

The project does not use:

- Stop-word removal.
- Lemmatization.
- Stemming.
- NLTK preprocessing.
- IP-address placeholders.

- URL/email placeholders.

This is important in viva: never claim these features exist.

## 4.11 What Happens If Preprocessing Is Removed?

For baseline models:

- Vocabulary becomes noisier.
- Uppercase/lowercase variants split across features.
- URLs and emails may become high-dimensional unique tokens.
- Similar tickets may look less similar.
- Model files may become larger.
- Generalization may decrease.

For production DistilBERT inference, the service already uses the tokenizer directly rather than calling `clean_ticket_text`. Transformers can use punctuation and natural phrasing as context. Aggressive baseline-style preprocessing can remove useful context.

---

## SECTION 5: BASELINE MODELS

### 5.1 Why Baselines Matter

A baseline answers: "Does the complex model provide enough improvement to justify its cost?" Without baselines, DistilBERT's score has no practical comparison.

### 5.2 TF-IDF

TF-IDF means Term Frequency-Inverse Document Frequency.

#### Intuition

- **Term frequency:** a word matters more if it appears in a ticket.
- **Inverse document frequency:** a word matters less if it appears in almost every ticket.

Simplified formula:

```
TF-IDF(term, document) = TF(term, document) * log(total documents / documents containing term)
```

Words such as "please" may have low importance because they appear everywhere. Phrases such as "admin rights" may have high importance because they are strongly associated with a smaller subset of tickets.

#### Actual Configuration

`build_tfidf_vectorizer()` uses:

- `max_features=50_000`
- `ngram_range=(1, 2)` for unigrams and bigrams
- `min_df=2`
- `max_df=0.95`
- `sublinear_tf=True`

Bigram support lets "admin rights" become a feature, not only admin and rights.

## 5.3 Logistic Regression

Despite its name, Logistic Regression is a classification algorithm.

### Mathematical Intuition

It calculates a weighted sum of input features and converts class scores into probabilities. During training it learns which TF-IDF features increase or decrease the likelihood of each class.

For example:

- "monitor" may strongly support Hardware.
- "mailbox" may support Storage.
- "purchase order" may support Purchase.

### Actual Configuration

- `C=2.0`
- `class_weight="balanced"`
- `max_iter=1000`
- `random_state=42`

`class_weight="balanced"` gives minority classes more influence during training.

### Strengths

- Strong performance on sparse text.
- Fast inference.
- Smaller artifact than DistilBERT.
- Feature weights can be inspected.
- Achieved 0.8616 test accuracy and 0.8622 macro F1.

### Weaknesses

- Limited understanding of context and word order beyond n-grams.
- Cannot naturally understand paraphrases.

- Depends heavily on training vocabulary.

## 5.4 Multinomial Naive Bayes

### Mathematical Intuition

Naive Bayes estimates:

$P(\text{class} \mid \text{words}) \propto P(\text{class}) * \text{product of } P(\text{word} \mid \text{class})$

It assumes features are conditionally independent given the class. This assumption is "naive," but the model often works well for text.

### Actual Configuration

- $\alpha=0.5$

Alpha smoothing prevents unseen terms from causing zero probability.

### Strengths

- Very fast.
- Simple.
- Works naturally with non-negative TF-IDF features.
- High macro precision in this run: 0.8802.

### Weaknesses

- Independence assumption is unrealistic.
- Minority-class recall can be weak.
- Actual test macro recall is 0.6814.
- Administrative rights recall is only 0.2159.

## 5.5 How Baseline Predictions Are Made

1. Clean ticket text.
2. Transform text using the saved TF-IDF vectorizer.
3. Pass sparse vector to the saved classifier.
4. Classifier returns a label.

The production API currently uses DistilBERT, not these baseline artifacts.

## 5.6 Saved Artifacts

| Artifact                    | Purpose                                                          |
|-----------------------------|------------------------------------------------------------------|
| models/tfidf_vectorizer.pkl | Learned vocabulary, IDF weights, and vectorization configuration |

| Artifact                       | Purpose                                              |
|--------------------------------|------------------------------------------------------|
| models/logistic_regression.pkl | Learned Logistic Regression coefficients and classes |
| models/naive_bayes.pkl         | Learned class priors and feature probabilities       |

The vectorizer must match the classifier. A classifier cannot correctly interpret a vector generated from a different vocabulary.

## 5.7 Baseline Metrics

| Model                   | Accuracy | Macro Precision | Macro Recall | Macro F1 | Weighted F1 |
|-------------------------|----------|-----------------|--------------|----------|-------------|
| Logistic Regression     | 0.8616   | 0.8507          | 0.8762       | 0.8622   | 0.8620      |
| Multinomial Naive Bayes | 0.7795   | 0.8802          | 0.6814       | 0.7336   | 0.7732      |

# SECTION 6: DISTILBERT DEEP DIVE

## 6.1 What DistilBERT Is

DistilBERT is a smaller, faster version of BERT produced through knowledge distillation. BERT is a Transformer encoder trained to understand language context. DistilBERT keeps much of BERT's language capability while using fewer layers.

Simple explanation:

DistilBERT reads a ticket while considering how every important word relates to the other words, then produces a contextual representation used for classification.

Technical explanation:

DistilBERT is a multi-layer Transformer encoder. Token embeddings pass through stacked self-attention and feed-forward blocks. The final hidden representation associated with the first sequence position is passed through a sequence-classification head to produce logits for eight labels.

## 6.2 Why Context Matters

In bag-of-words models, "access" is simply a feature. In a contextual model:

- "Need access to Confluence" means permission request.
- "Access card is broken" may concern a device/access mechanism.
- "Database access is slow" may imply a system issue.

Self-attention lets the model relate "access" to nearby and distant words.

## 6.3 Tokenization

`DistilBertTokenizerFast` uses WordPiece-like subword tokenization. Unknown or rare words can be split into smaller known pieces.

Conceptual example:

```
"administrative" -> ["admin", "##istrative"] (illustrative only)
```

The exact split depends on the tokenizer vocabulary. Tokenization produces:

- `input_ids`: integer token IDs.
- `attention_mask`: 1 for real tokens and 0 for padding.

During training, labels are integer IDs from 0 to 7.

## 6.4 Embeddings

Token IDs are not meaningful by themselves. The model converts them into dense vectors. Position information tells the model token order. These vectors pass through the Transformer layers.

## 6.5 Self-Attention

Self-attention calculates how strongly each token should consider other tokens.

Each token produces:

- **Query**: what information it seeks.
- **Key**: what information it offers.
- **Value**: the information content.

Simplified attention:

```
Attention(Q, K, V) = softmax(QK^T / sqrt(d))V
```

The softmax creates attention weights. Multi-head attention learns several relationship types in parallel.

## 6.6 Classification Head

The model configuration declares `DistilBertForSequenceClassification` and eight labels. The classification head converts the final sequence representation into eight logits:

```
[Access score, Administrative-rights score, HR score, ... Storage score]
```

Softmax at inference converts logits into probabilities summing to 1.

## 6.7 Training Configuration

- Base model: `distilbert-base-uncased`

- Classes: 8
- Seed: 42
- Split: stratified 80/10/10
- Maximum length: 128
- Batch size: 16
- Epochs requested and completed: 3
- Learning rate:  $2e-5$
- Weight decay: 0.01
- Warmup: 10% of calculated training steps
- Mixed precision: enabled in recorded CUDA run
- Evaluation: every epoch
- Checkpointing: every epoch
- Best-model metric: validation macro F1
- Early-stopping patience: 2
- Recorded GPU: NVIDIA GeForce RTX 4060 Ti
- Best checkpoint: `models/distilbert/checkpoint-7176`

## 6.8 Reproducibility

`seed_everything()` seeds:

- Python random.
- NumPy.
- PyTorch CPU.
- PyTorch CUDA.
- Hugging Face Transformers.

It also requests deterministic cuDNN behavior. Reproducibility can still be influenced by hardware and library implementation details, but this is a strong practical setup.

## 6.9 Training Flow

1. Load and normalize dataset.
2. Create stratified split.
3. Sort labels and create `label2id/id2label`.
4. Tokenize all splits.
5. Create `TicketTextDataset` objects.

6. Load pre-trained DistilBERT with eight output labels.
7. Configure `TrainingArguments`.
8. Train using Hugging Face `Trainer`.
9. Evaluate validation macro F1 each epoch.
10. Save checkpoints and restore the best model.
11. Save final model, tokenizer, and label mapping.
12. Predict validation and test splits.
13. Generate aggregate metrics, per-class metrics, and confusion matrix.

## 6.10 Inference Flow

When a user enters a ticket:

1. The API service receives text.
2. Tokenizer truncates, pads, and returns PyTorch tensors.
3. Tensors move to the selected device.
4. `torch.no_grad()` saves memory because gradients are unnecessary.
5. The model outputs logits.
6. Softmax produces probabilities.
7. `torch.max()` selects confidence and label ID.
8. `id2label` converts ID to category.

## 6.11 Saved DistilBERT Artifacts

| Artifact                           | Purpose                                                                  |
|------------------------------------|--------------------------------------------------------------------------|
| <code>model.safetensors</code>     | Trained model weights                                                    |
| <code>config.json</code>           | Architecture and label configuration                                     |
| <code>tokenizer.json</code>        | Tokenizer vocabulary and rules                                           |
| <code>tokenizer_config.json</code> | Tokenizer settings                                                       |
| <code>label_mapping.json</code>    | Explicit label-to-ID mapping                                             |
| <code>training_args.bin</code>     | Saved training arguments                                                 |
| <code>checkpoint - *</code>        | Intermediate model, optimizer, scheduler, RNG, scaler, and trainer state |

## 6.12 DistilBERT Results

| Metric          | Validation | Test   |
|-----------------|------------|--------|
| Accuracy        | 0.8794     | 0.8878 |
| Macro Precision | 0.8820     | 0.8922 |
| Macro Recall    | 0.8701     | 0.8847 |
| Macro F1        | 0.8757     | 0.8883 |
| Weighted F1     | 0.8793     | 0.8877 |

DistilBERT improves test accuracy by 2.62 percentage points over Logistic Regression and macro F1 by 0.0261.

---

## SECTION 7: ENTITY EXTRACTION

### 7.1 What spaCy Does Here

spaCy provides the NLP pipeline infrastructure. The project creates `spacy.blank("en")`, meaning it does not download or load a statistical English model. It then adds an `EntityRuler`.

### 7.2 EntityRuler

`EntityRuler` identifies entities using patterns. This makes extraction:

- Deterministic.
- Explainable.
- Easy to configure.
- Fast to start.
- Independent of a labeled NER training dataset.

The configuration sets:

- `overwrite_ents=True`
- `phrase_matcher_attr="LOWER"` for case-insensitive phrases
- `validate=True`

### 7.3 Pattern Configuration

Patterns are stored in `config/entity_patterns.json`. Each pattern has:

- `label`
- `pattern`
- `id`

Example:

```
{"label": "SOFTWARE", "pattern": "Outlook", "id": "software_outlook"}
```

Strict error codes use token regex patterns rather than phrase strings.

## 7.4 Entity Types

### SOFTWARE

Named applications, platforms, tools, or operating systems.

Examples: Oracle, Confluence, Windows, Outlook, Excel, Visual Studio, VS Code, Chrome, Active Directory, Teams, SAP, Edge, Word, Java, Slack, Zoom, Acrobat.

### DEVICE

Physical endpoints, peripherals, and access media.

Examples: access card, ID card, badge, laptop, monitor, phone, printer, keyboard, mouse, workstation, scanner, headset, router, docking station, tablet.

### ERROR\_CODE

Machine-readable error identifiers. Rules are intentionally strict.

Patterns include:

- Hex codes such as 0x80070005.
- Prefixed IDs such as ERR-123.
- HTTP 4xx/5xx phrases such as HTTP 500.
- Windows KB IDs such as KB123456.

Generic text such as "an error occurred" is not an ERROR\_CODE.

### SYSTEM

Infrastructure, accounts, storage, permissions, or business-system resources.

Examples: mailbox, folder, account, permissions, storage, backup, network, drive, database, domain, portal, admin rights, payroll, file share, server.

### LOCATION

Workplace/site-related locations.

Examples: meeting room, conference room, data center, floor, site, room, remote, building, branch, desk, reception, Delhi.

## 7.5 Match Behavior

When a pattern matches, spaCy creates an entity span. `EntityExtractor.extract()` converts each span to:

```
EntityMatch(  
    text="Outlook",  
    label="SOFTWARE",  
    start_char=0,  
    end_char=7,  
    pattern_id="software_outlook"  
)
```

Longer phrases win naturally in tested cases. "access card" is extracted instead of separate "card", and "conference room" instead of "room".

## 7.6 No-Match Behavior

- Blank text returns [ ].
- Nonblank text with no configured matches also returns [ ].
- The API groups matches into a dictionary; no matches become {}.
- The dashboard displays `None` for each known entity group that is absent.

## 7.7 Limitations

- Unknown software or device names are not recognized until patterns are added.
- Ambiguous phrases may be labeled without deeper context.
- No statistical NER metrics are available.
- Pattern changes require configuration deployment but not model retraining.

---

# SECTION 8: PRIORITY ENGINE

## 8.1 Architecture

Priority is independent from the classifier. It answers "How urgent does the text appear?" while classification answers "What category is this?"

Files:

- `src/priority/rules.py`
- `src/priority/engine.py`

Output:

```
PriorityResult(priority="High", matched_rule="cannot login")
```

## 8.2 Matching Order

The dictionary preserves this order:

1. Critical

2. High
3. Medium
4. Low

The engine returns immediately on the first match. Therefore a Critical phrase overrides a Medium phrase in the same ticket.

Example:

```
"Production down and need installation support"
```

Matches both `production down` and `installation`, but returns Critical because Critical is checked first.

## 8.3 Critical Rules

| Canonical Rule                  | Why                                 | Example                           | Output                        |
|---------------------------------|-------------------------------------|-----------------------------------|-------------------------------|
| <code>server down</code>        | Server outage may affect many users | "Production server is down"       | Critical / server down        |
| <code>production down</code>    | Business production outage          | "Production is down"              | Critical / production down    |
| <code>outage</code>             | Explicit broad interruption         | "There is a network outage"       | Critical / outage             |
| <code>system unavailable</code> | Complete service unavailability     | "System unavailable for everyone" | Critical / system unavailable |
| <code>data loss</code>          | Potentially irreversible damage     | "We experienced data loss"        | Critical / data loss          |

Aliases include `"server is down"`, `"server went down"`, `"server unavailable"`, `"production is down"`, `"prod down"`, and `"prod is down"`.

## 8.4 High Rules

| Canonical Rule                       | Why                                 | Example                   | Output                         |
|--------------------------------------|-------------------------------------|---------------------------|--------------------------------|
| <code>cannot login</code>            | User cannot access required service | "Unable to log in"        | High / cannot login            |
| <code>access denied</code>           | Explicit blocked access             | "Access denied to portal" | High / access denied           |
| <code>vpn not working</code>         | Remote connectivity failure         | "VPN down"                | High / vpn not working         |
| <code>email not working</code>       | Core communication unavailable      | "Outlook not working"     | High / email not working       |
| <code>application unavailable</code> | Required application cannot be used | "Application unavailable" | High / application unavailable |

The canonical matched rule is returned even when an alias matched. This is why "unable to log in" returns "cannot login".

## 8.5 Medium Rules

- installation
- software request
- upgrade
- configuration

These usually require action but do not necessarily indicate an outage.

## 8.6 Low Rules

- information request
- documentation
- inquiry

Aliases for documentation include user guide, guide, and manual.

## 8.7 Default Behavior

If no rule matches:

```
{"priority": "Medium", "matched_rule": null}
```

Medium is selected as a conservative operational default: not urgent enough to claim High/Critical, but not automatically Low.

## 8.8 Error Behavior

- Non-string input raises `TypeError`.
- Empty strings are valid strings and simply default to Medium.
- An empty `PriorityResult.priority` raises `ValueError`.

---

# SECTION 9: ROUTING ENGINE

## 9.1 Mapping

| Category   | Team                   |
|------------|------------------------|
| Hardware   | Hardware Team          |
| HR Support | HR Team                |
| Access     | Access Management Team |

| Category              | Team                       |
|-----------------------|----------------------------|
| Storage               | Storage Team               |
| Purchase              | Procurement Team           |
| Administrative rights | System Administration Team |
| Internal Project      | Internal Projects Team     |
| Miscellaneous         | Service Desk Team          |

## 9.2 Why Every Route Exists

- Hardware Team has ownership of physical devices.
- HR Team owns HR-related processes and systems.
- Access Management Team handles permissions and authentication.
- Storage Team handles mailbox, file-space, and storage capacity.
- Procurement Team handles purchases.
- System Administration Team handles elevated privileges.
- Internal Projects Team handles project setup and codes.
- Service Desk Team is the broad destination for Miscellaneous tickets.

## 9.3 Internal Logic

1. `_normalize_category()` verifies the value is a string.
2. Leading/trailing whitespace is removed.
3. Blank category raises `UnsupportedCategoryError`.
4. `get_team_for_category()` performs exact lookup.
5. Unknown category raises a descriptive error listing supported categories.
6. Valid result is returned as `RouteResult`.

Routing is case-sensitive after trimming. "Hardware" works; "hardware" is unsupported.

## 9.4 Why Routing Is Separate From Classification

- Classification can improve without changing business rules.
  - Teams can change without retraining the model.
  - Unknown model labels become visible immediately.
  - Routing logic is easy to unit test.
  - Separation avoids embedding organizational policy inside model weights.
-

## SECTION 10: FASTAPI BACKEND

### 10.1 File Responsibilities

| File                    | Responsibility                              |
|-------------------------|---------------------------------------------|
| src/api/app.py          | FastAPI app and endpoint functions          |
| src/api/schemas.py      | Pydantic request/response models            |
| src/api/dependencies.py | Dependency-provider bridge                  |
| src/api/services.py     | Loads components and orchestrates workflows |
| src/api/__init__.py     | Package marker                              |

### 10.2 Service Startup and Singleton

`get_ticket_analysis_service()` has `@lru_cache(maxsize=1)`. The first dependency request constructs `TicketAnalysisService`; later requests reuse it.

At construction it:

1. Selects CUDA or CPU.
2. Loads label mapping.
3. Loads saved tokenizer.
4. Loads saved DistilBERT model.
5. Moves model to device and sets evaluation mode.
6. Creates entity extractor.
7. Creates priority engine.
8. Creates router.
9. Creates prediction repository and initializes database tables.

This prevents expensive model reload on every request.

### 10.3 GET /health

**Request:** none.

**Response:**

```
{"status": "healthy"}
```

**Logic:** returns a fixed health response. It does not deeply verify model or database availability.

### 10.4 GET /metrics

**Response example:**

```
{
  "model_name": "distilbert-base-uncased",
  "number_of_classes": 8,
  "available_entities": ["DEVICE", "ERROR_CODE", "LOCATION", "SOFTWARE",
"SYSTEM"],
  "supported_priorities": ["Critical", "High", "Medium", "Low"]
}
```

**Logic:** obtains runtime metadata. If model mapping cannot load, `ModelLoadError` becomes HTTP 503.

Note: this endpoint returns model/service metadata, not live accuracy metrics.

## 10.5 POST /predict

**Request:**

```
{"ticket_text": "Need admin rights for Visual Studio"}
```

**Response shape:**

```
{"category": "Administrative rights", "confidence": 0.93}
```

**Logic:** calls `service.predict()`. Actual category/confidence depend on model inference.

**Errors:** Pydantic returns 422 for missing/empty text. Model-loading/runtime failures are not specially converted here.

## 10.6 POST /entities

**Request:**

```
{"ticket_text": "Outlook not working on laptop"}
```

**Response:**

```
{"entities": {"SOFTWARE": ["Outlook"], "DEVICE": ["laptop"]}}
```

**Logic:** calls `service.extract_entities()`.

## 10.7 POST /priority

**Request:**

```
{"ticket_text": "Production server is down"}
```

**Response:**

```
{"priority": "Critical", "matched_rule": "server down"}
```

**Logic:** calls the same priority engine used by `/analyze`. The `matched_rule` field is preserved.

## 10.8 POST /route

**Request:**

```
{"category": "Hardware"}
```

**Response:**

```
{"category": "Hardware", "assigned_team": "Hardware Team"}
```

**Error:** unsupported categories become HTTP 400 with the descriptive router error.

## 10.9 POST /analyze

This is the complete workflow endpoint.

**Request:**

```
{"ticket_text": "I am unable to log in to Outlook on my laptop"}
```

**Response shape:**

```
{
  "category": "Access",
  "confidence": 0.92,
  "entities": {"SOFTWARE": ["Outlook"], "DEVICE": ["laptop"]},
  "priority": "High",
  "matched_rule": "cannot login",
  "assigned_team": "Access Management Team"
}
```

**Internal order:**

1. Predict.
2. Extract entities.
3. Assign priority.
4. Route predicted category.
5. Persist result.
6. Return response.

Unsupported predicted category becomes HTTP 400. If persistence fails after earlier steps, the request fails rather than returning an unpersisted success.

## 10.10 GET /history

**Optional query:** limit, default 50.

**Response:** recent predictions, newest first.

The repository clamps the limit to 1 through 500.

## 10.11 GET /analytics

**Response:**

```
{
  "total_predictions": 12,

```

```
"category_distribution": {"Access": 4, "Hardware": 8},  
"priority_distribution": {"High": 5, "Medium": 7}  
}
```

Counts are generated from persisted records, not training data.

## 10.12 OpenAPI

FastAPI generates:

- Swagger UI: /docs
- ReDoc: /redoc
- OpenAPI JSON: /openapi.json

## 10.13 Run Command

```
uvicorn src.api.app:app --reload
```

---

# SECTION 11: DATABASE

## 11.1 Why SQLite

SQLite stores data in a local file and requires no separate server. It is suitable for a student project, demo, prototype, and single-instance deployment.

## 11.2 Why SQLAlchemy

SQLAlchemy provides:

- ORM mapping from Python classes to database tables.
- Structured query construction.
- Session and transaction management.
- Easier migration to another relational database later.

## 11.3 Database Configuration

src/database/database.py defines:

- DATABASE\_PATH = data/ticket\_routing.db
- DATABASE\_URL = sqlite:///...
- SQLAlchemy engine.
- SessionLocal.
- init\_database().

`check_same_thread=False` allows SQLite usage across FastAPI request threads.

## 11.4 ticket\_predictions Table

| Column                          | Meaning                          |
|---------------------------------|----------------------------------|
| <code>id</code>                 | Auto-increment primary key       |
| <code>ticket_text</code>        | Original submitted text          |
| <code>predicted_category</code> | DistilBERT category              |
| <code>confidence_score</code>   | Probability of selected category |
| <code>entities_json</code>      | Entities serialized as JSON text |
| <code>priority</code>           | Rule-based priority              |
| <code>assigned_team</code>      | Routing result                   |
| <code>created_at</code>         | UTC creation timestamp           |

Category, priority, and timestamp columns are indexed where configured to support common queries.

## 11.5 Repository Methods

### **`save_prediction(...)`**

- Sorts entity keys during JSON serialization.
- Opens a session.
- Adds row.
- Commits transaction.
- Refreshes object to receive ID.
- Returns ORM object.

### **`get_recent_predictions(limit=50)`**

- Clamps limit between 1 and 500.
- Orders by `created_at` descending, then ID descending.
- Serializes rows to dictionaries.

### **`get_prediction_count()`**

Runs SQL `COUNT(id)`.

### **`get_category_distribution()`**

Groups persisted records by predicted category.

## get\_priority\_distribution()

Groups persisted records by priority.

## 11.6 Automatic Table Creation

PredictionRepository() calls init\_database() by default.

Base.metadata.create\_all() creates missing tables. It does not perform schema migrations.

## 11.7 Persistence Limitations

- matched\_rule is returned by the API but is not stored.
  - Entities are JSON text, not normalized relational rows.
  - No user ID, model version, feedback, corrected label, request latency, or audit actor is stored.
  - SQLite is not ideal for high-write distributed deployments.
- 

# SECTION 12: STREAMLIT DASHBOARD

## 12.1 Files

| File                    | Responsibility                                     |
|-------------------------|----------------------------------------------------|
| dashboard/app.py        | Page layout, tabs, health check, analysis workflow |
| dashboard/components.py | Reusable display panels                            |
| dashboard/charts.py     | Plotly chart builders                              |
| dashboard/utils.py      | HTTP calls, parsing, report loading, formatting    |

## 12.2 Sidebar

The sidebar contains:

- Editable FastAPI URL.
- System status.

Default URL comes from BACKEND\_URL, falling back to http://127.0.0.1:8000. Docker Compose sets BACKEND\_URL=http://backend:8000.

## 12.3 Analyze Ticket Tab

User action:

1. Enter text.

2. Click Analyze Ticket.

Dashboard behavior:

- Rejects empty input with a warning.
- Calls `/analyze`, `/predict`, `/entities`, and `/priority`.
- Parses responses.
- Shows Prediction, Entity Extraction, Priority, Routing, and Full Analysis panels.

An optional `demo_ticket` and `auto_analyze` query-parameter path exists to support deterministic screenshot/demo capture.

## 12.4 Components

- `render_status()`: green healthy or red offline message.
- `render_prediction_panel()`: category and percentage confidence.
- `render_entities_panel()`: displays SOFTWARE, DEVICE, SYSTEM, LOCATION, ERROR\_CODE in fixed order.
- `render_priority_panel()`: displays priority and matched rule; None appears as Default.
- `render_routing_panel()`: displays assigned team.

## 12.5 Analytics Tab

### Model Comparison

Reads:

- `reports/model_baselines.md`
- `reports/model_distilbert.md`

It parses Markdown tables and displays test accuracy, macro precision, macro recall, and macro F1.

### Class Distribution

Reads `reports/eda/class_distribution.csv` and creates a horizontal bar chart.

### Priority and Routing Distribution

These charts currently use hard-coded sample/demo values in `dashboard/charts.py`. They do not use `/analytics`.

## 12.6 Error Handling

`DashboardAPIError` wraps:

- Connection failures.

- HTTP errors.
- Invalid JSON.
- Invalid top-level response shape.

The dashboard catches API errors during analysis and displays a compact message. Health failures display Offline.

## 12.7 Run Command

```
streamlit run dashboard/app.py
```

---

# SECTION 13: DOCKER

## 13.1 Why Containers

Containers package application code and dependencies into a reproducible runtime. They reduce "works on my machine" problems and simplify starting multiple services.

## 13.2 Dockerfile

The Dockerfile:

1. Starts from `python:3.11-slim`.
2. Disables `.pyc` generation and enables unbuffered output.
3. Sets `/app` as working directory.
4. Installs `curl` for health checks.
5. Copies `requirements.txt` first for layer caching.
6. Installs dependencies.
7. Copies the project.
8. Creates `data`, `models`, and `reports`.
9. Exposes ports 8000 and 8501.
10. Defaults to running Uvicorn backend.

## 13.3 Docker Compose Services

### Backend

- Builds from project Dockerfile.
- Runs Uvicorn on `0.0.0.0:8000`.
- Maps host 8000 to container 8000.

- Mounts data, models, and reports.
- Uses `curl` healthcheck on `/health`.

## Dashboard

- Builds the same image.
- Overrides command to run Streamlit on `0.0.0.0:8501`.
- Maps host `8501` to container `8501`.
- Mounts the same directories.
- Sets `BACKEND_URL=http://backend:8000`.
- Waits until backend healthcheck succeeds.

## 13.4 Service Communication

Inside Docker Compose, service names act as DNS names. The dashboard reaches the backend using hostname `backend`, not `localhost`. Inside the dashboard container, `localhost` would refer to the dashboard container itself.

## 13.5 Volumes

- `./data:/app/data`: preserves SQLite database.
- `./models:/app/models`: makes trained model available.
- `./reports:/app/reports`: makes analytics reports available.

## 13.6 Startup

```
docker compose up --build
```

Expected URLs:

- API docs: `http://localhost:8000/docs`
- Dashboard: `http://localhost:8501`

Successful Docker execution is **UNVERIFIED - REQUIRES REPOSITORY VALIDATION**.

---

# SECTION 14: TESTING

## 14.1 Current Result

```
58 passed in 16.41s
```

This proves the tested contracts passed on the validation machine. It does not prove absence of all bugs or production scalability.

## 14.2 Test Files

### **tests/test\_preprocessing.py - 3 tests**

- URL/email removal.
- Preservation of technical tokens such as hex codes and Windows paths.
- Missing-value handling in a Pandas series.

Failure means normalization behavior changed or became unsafe for technical text.

### **tests/test\_base\_lines.py - 2 tests**

- Stratified split preserves all classes and expected sizes on sample data.
- Classifier evaluation returns per-class metrics and confusion matrix.

Failure means model-training utilities or metric contracts are broken.

### **tests/test\_distilbert.py - 2 tests**

- Evaluation builds per-class metrics and confusion matrix from logits.
- Reproducibility seeding runs.

These tests do not retrain DistilBERT; they test helper behavior.

### **tests/test\_ner.py - 4 tests**

- Extracts software, device, location, and error code.
- Prefers longer phrases.
- Blank text returns no entities.
- Generic error text is not falsely labeled as an error code.

Failure means pattern behavior or extraction safety changed.

### **tests/test\_priority.py - 10 tests**

- Critical, High, Medium, and Low cases.
- Critical precedence.
- Alias handling.
- Regression for "unable to log in".
- Default Medium behavior.
- Case-insensitivity.
- Result validation.
- Non-string input.

Failure means priority rules, hierarchy, or matched-rule preservation changed.

## **tests/test\_routing.py - 6 tests**

- Every configured mapping.
- Unknown, blank, and non-string category behavior.
- `RouteResult` validation.
- Read-only mapping helpers.

Failure means business routing logic or validation changed.

## **tests/test\_api.py - 10 tests**

Uses a stub service so endpoint contracts can be tested without loading the real DistilBERT model.

Tests health, metrics, prediction, entities, priority, routing, analysis, matched-rule consistency, history, and analytics.

Failure means an endpoint's request/response contract changed.

## **tests/test\_database.py - 4 tests**

Uses an in-memory SQLite database.

- Table creation.
- Saving prediction and JSON entities.
- Recent-query ordering and UTC timestamp.
- Count/category/priority analytics.

Failure means persistence behavior changed.

## **tests/test\_dashboard\_utils.py - 8 tests**

- Prediction and entity parsing.
- Priority matched-rule preservation.
- Invalid response rejection.
- Markdown table parsing.
- URL normalization.
- Confidence formatting.
- Environment URL loading.
- Chart helper output types.

Failure means dashboard parsing or analytics rendering could break.

## **tests/test\_config.py - 4 tests**

- Local backend URL default.
- Environment override.

- URL normalization.
- Docker Compose commands, ports, volumes, healthcheck, environment, and dependency.

Failure means deployment configuration drifted from expected behavior.

## tests/test\_integration.py - 5 tests

Uses a deterministic classifier stub but real NER, priority, routing, FastAPI, and an in-memory repository.

- Prediction endpoint workflow.
- Full analysis persistence.
- History and analytics after analyses.
- Routing, priority, and entity integration.
- Priority matched-rule consistency across endpoints.

Failure means modules no longer cooperate correctly.

## 14.3 Why Stubs Are Used

Loading and running a full transformer in every unit test is slow and makes tests depend on large artifacts and hardware. Stubs isolate API contracts. Real helper logic is tested separately, and integration tests combine most business modules.

## 14.4 What Is Not Fully Tested

- Full real DistilBERT inference inside integration tests.
- Load/performance testing.
- Concurrent database writes.
- Docker runtime.
- Browser-level interactive dashboard automation as part of pytest.
- Security testing.

---

# SECTION 15: ERROR SCENARIOS

## 15.1 Dataset and EDA

| Failure                  | Behavior          | Recovery                                  |
|--------------------------|-------------------|-------------------------------------------|
| Dataset path missing     | FileNotFoundError | Place CSV at expected path or pass --data |
| Required columns missing | ValueError        | Restore Document and Topic_group          |

| Failure               | Behavior                                       | Recovery                |
|-----------------------|------------------------------------------------|-------------------------|
| Fewer than two labels | Baseline loader raises <code>ValueError</code> | Validate dataset labels |

## 15.2 Preprocessing

- Missing value becomes empty string.
- Unexpected object is converted to string unless Pandas considers it missing.
- Excessive filtering could remove useful characters; regression tests protect known technical tokens.

## 15.3 Model Loading

| Failure                                  | Behavior                    | Recovery                               |
|------------------------------------------|-----------------------------|----------------------------------------|
| <code>label_mapping.json</code> missing  | <code>ModelLoadError</code> | Restore model directory/artifact       |
| Model/tokenizer files missing or corrupt | Transformers loading error  | Restore or retrain model               |
| GPU unavailable                          | Service uses CPU            | Accept slower inference or provide GPU |
| Incompatible library/artifact versions   | Loading/runtime error       | Recreate compatible environment        |

## 15.4 NER

| Failure                 | Behavior                             | Recovery                                         |
|-------------------------|--------------------------------------|--------------------------------------------------|
| Pattern file missing    | <code>FileNotFoundError</code>       | Restore <code>config/entity_patterns.json</code> |
| Pattern JSON not a list | <code>ValueError</code>              | Fix JSON structure                               |
| Invalid spaCy pattern   | EntityRuler validation/startup error | Correct pattern                                  |
| No match                | Empty entity list/dictionary         | Add pattern if required                          |

## 15.5 Priority

- Non-string input raises `TypeError`.
- No rule match returns `Medium`, not an error.
- Ambiguous ticket returns the first highest-priority matching rule.
- Missing desired synonym means default or another rule may be selected; add alias and test.

## 15.6 Routing

- Unknown, blank, or non-string category raises `UnsupportedCategoryError`.
- `/route` and `/analyze` convert unsupported categories to HTTP 400.
- Recovery: align model labels and routing configuration.

## 15.7 API Validation

- Missing or empty `ticket_text`: HTTP 422 from Pydantic.
- Invalid confidence outside 0 to 1: response-model validation failure.
- Unsupported route category: HTTP 400.
- Model metadata loading failure in `/metrics`: HTTP 503.
- Other unhandled service errors: HTTP 500.

## 15.8 Database

- Missing database file is normally recovered because tables are created automatically.
- Unwritable `data/` directory causes initialization or commit failure.
- Corrupt database causes SQLAlchemy/SQLite errors.
- Schema changes are not migrated automatically; `create_all` only creates missing tables.

Recovery:

1. Back up database.
2. Verify directory permissions.
3. Inspect schema.
4. Introduce migrations before production schema evolution.

## 15.9 Dashboard

- API unavailable: health shows Offline; analysis catches `DashboardAPIError`.
- Empty input: warning shown.
- Invalid JSON or response shape: readable error shown.
- Missing report files: analytics displays informational fallback.
- Wrong `BACKEND_URL`: update sidebar or environment.

## 15.10 Docker

- Backend cannot start if model artifacts are absent.
- Healthcheck fails if backend is slow or unhealthy.

- Dashboard waits for healthy backend.
  - Volume permissions can block SQLite writes.
  - Dependency installation can fail without network access.
- 

## SECTION 16: IF-ELSE DECISION TREES

### 16.1 Full Request

```
IF request ticket_text is missing or empty
THEN FastAPI returns HTTP 422
ELSE predict category

IF predicted category has routing mapping
THEN continue
ELSE return HTTP 400

IF entity patterns match
THEN group matched entities by label
ELSE entities = {}

IF a Critical priority rule matches
THEN priority = Critical and stop priority search
ELSE IF a High rule matches
THEN priority = High and stop
ELSE IF a Medium rule matches
THEN priority = Medium and stop
ELSE IF a Low rule matches
THEN priority = Low and stop
ELSE priority = Medium and matched_rule = null

IF persistence succeeds
THEN return full analysis
ELSE request fails
```

### 16.2 Routing Tree

```
IF category == "Hardware" THEN "Hardware Team"
ELSE IF category == "HR Support" THEN "HR Team"
ELSE IF category == "Access" THEN "Access Management Team"
ELSE IF category == "Storage" THEN "Storage Team"
ELSE IF category == "Purchase" THEN "Procurement Team"
ELSE IF category == "Administrative rights" THEN "System Administration Team"
ELSE IF category == "Internal Project" THEN "Internal Projects Team"
ELSE IF category == "Miscellaneous" THEN "Service Desk Team"
ELSE raise UnsupportedCategoryError
```

### 16.3 Device Selection

```
IF torch.cuda.is_available()
THEN use CUDA
ELSE use CPU
```

## 16.4 Dashboard URL

```
IF BACKEND_URL environment variable exists
THEN normalize and use it
ELSE use http://127.0.0.1:8000
```

## 16.5 Dashboard Analysis

```
IF Analyze Ticket clicked OR demo auto-analysis enabled
THEN check text

IF text is blank
THEN show warning
ELSE call backend endpoints

IF backend call fails
THEN show compact error
ELSE display result panels
```

## 16.6 History Limit

```
IF limit < 1 THEN use 1
ELSE IF limit > 500 THEN use 500
ELSE use requested limit
```

## 16.7 Entity Display

```
FOR each label in SOFTWARE, DEVICE, SYSTEM, LOCATION, ERROR_CODE:
  IF values exist THEN display comma-separated values
  ELSE display None
```

---

# SECTION 17: DESIGN DECISIONS

## 17.1 Why DistilBERT?

DistilBERT provides contextual understanding and achieved the best verified test results. It is smaller than full BERT and therefore more practical for a student deployment.

Alternatives:

- Full BERT: potentially stronger but larger/slower.
- Logistic Regression: faster and simpler, but lower verified performance.
- LSTM: not implemented; would require a separate sequence pipeline and likely more task-specific training effort.

## 17.2 Why FastAPI?

- Native Pydantic validation.
- Automatic OpenAPI documentation.

- Clear dependency injection.
- Strong typing.
- Simple Uvicorn deployment.

Alternative Flask is flexible and lightweight, but would require more manual schema validation and documentation setup.

## 17.3 Why SQLite?

- No separate database server.
- Easy demonstration and portability.
- Sufficient for local prototype history and analytics.

Alternatives:

- PostgreSQL for concurrent production workloads and stronger operational features.
- MongoDB for flexible document storage, though the data and analytics here fit a relational structure well.

## 17.4 Why Streamlit?

- Rapid Python-based dashboard development.
- Easy integration with Pandas and Plotly.
- Suitable for an ML demonstration.

Alternatives:

- React/Next.js for greater UI control and production frontend architecture.
- Gradio for model demos with simpler workflows.

## 17.5 Why EntityRuler?

- No labeled NER training dataset exists.
- Patterns are explainable and easy to extend.
- Strict error-code handling reduces false positives.

Alternative trained NER could generalize beyond known terms, but requires quality annotations and evaluation.

## 17.6 Why Rule-Based Priority?

The dataset contains category labels but no priority labels. A supervised priority model would require invented labels or another dataset. Rules provide deterministic, auditable behavior.

## 17.7 Why Separate Priority and Routing?

Priority and routing solve different business questions:

- Priority: how urgent?
- Routing: which team?

Separating them makes policy changes safer and tests clearer.

## 17.8 Why Macro F1?

Macro F1 gives equal importance to every class. With a 7.7369 imbalance ratio, accuracy alone can hide poor minority-class performance.

## 17.9 Why Stratified Splitting?

Stratification preserves category proportions across train, validation, and test sets, reducing the chance that minority classes disappear or become poorly represented.

---

# SECTION 18: VIVA PREPARATION

The answers below are deliberately phrased as answers you can speak. Adjust wording naturally, but do not change technical facts.

## 18.1 Beginner Questions

### 1. What is the title of your project?

My project is an automated IT support ticket classification and routing system using Natural Language Processing. It classifies ticket text, extracts entities, assigns priority, routes the ticket, persists the result, and exposes the workflow through FastAPI and Streamlit.

### 2. What problem does it solve?

It reduces manual service-desk triage. Instead of requiring a person to read and route every ticket, the system performs an initial automated analysis.

### 3. What is the input?

The main input is a non-empty natural-language IT support ticket string.

### 4. What is the main output?

The full output includes predicted category, confidence, extracted entities, priority, matched priority rule, and assigned support team.

### 5. How many classes are there?

There are eight classes: Access, Administrative rights, HR Support, Hardware, Internal Project, Miscellaneous, Purchase, and Storage.

## **6. How many tickets are in the dataset?**

The dataset has 47,837 tickets.

## **7. What are the dataset columns?**

Document contains ticket text and Topic\_group contains the target category.

## **8. Which model is the final classifier?**

The final production classifier is fine-tuned DistilBERT base uncased.

## **9. Which baseline models were trained?**

TF-IDF with Logistic Regression and TF-IDF with Multinomial Naive Bayes.

## **10. Was LSTM implemented?**

No. LSTM was not implemented, trained, or evaluated in the repository.

## **11. Why did you use NLP?**

The source data is free-text written by users. NLP converts this unstructured text into categories and structured metadata.

## **12. What is routing?**

Routing maps the predicted category to the support team responsible for that category.

## **13. What is priority assignment?**

Priority assignment examines ticket text for configured urgency phrases and returns Critical, High, Medium, or Low.

## **14. What is entity extraction?**

Entity extraction identifies useful technical terms such as Outlook, laptop, mailbox, data center, or HTTP 500.

## **15. What database is used?**

SQLite, accessed through SQLAlchemy.

## **16. What backend framework is used?**

FastAPI with Uvicorn.

## **17. What dashboard framework is used?**

Streamlit, with Plotly charts.

## **18. What is Docker used for?**

Docker configuration packages the backend and dashboard into repeatable container services.

### **19. What is the train-validation-test split?**

It is a seeded stratified 80/10/10 split: 38,269 training, 4,784 validation, and 4,784 test rows.

### **20. Why is a validation set needed?**

It supports model selection and early-stopping decisions without using the final test set.

### **21. Why is a test set needed?**

It provides a held-out estimate of final model performance.

### **22. What is accuracy?**

Accuracy is the fraction of all predictions that are correct.

### **23. What is precision?**

Precision measures how many predictions for a class were actually correct.

### **24. What is recall?**

Recall measures how many true items of a class the model successfully found.

### **25. What is F1-score?**

F1 is the harmonic mean of precision and recall.

### **26. What is macro F1?**

Macro F1 calculates F1 separately for each class and averages them equally. It is useful for imbalanced datasets.

### **27. What is the DistilBERT test accuracy?**

0.8878, or 88.78%.

### **28. What is the DistilBERT test macro F1?**

0.8883.

### **29. What is the Logistic Regression test accuracy?**

0.8616, or 86.16%.

### **30. What is the Naive Bayes test accuracy?**

0.7795, or 77.95%.

### **31. Why not report only accuracy?**

The dataset is imbalanced, so accuracy can hide poor minority-class performance.

### **32. What is the largest category?**

Hardware, with 13,617 tickets.

### **33. What is the smallest category?**

Administrative rights, with 1,760 tickets.

### **34. What is the class imbalance ratio?**

7.7369, calculated as largest class size divided by smallest class size.

### **35. What happens when no entity matches?**

The extractor returns no matches, the API entity dictionary may be empty, and the dashboard displays None for absent entity groups.

### **36. What happens when no priority rule matches?**

The engine returns Medium priority with no matched rule.

### **37. What happens for an unknown routing category?**

The router raises `UnsupportedCategoryError`, and API route/analyze handling converts it to HTTP 400.

### **38. Where are NER patterns stored?**

`config/entity_patterns.json`.

### **39. Where is the database stored?**

`data/ticket_routing.db`.

### **40. Which endpoint performs the complete workflow?**

`POST /analyze`.

## **18.2 Intermediate Questions**

### **41. Why was DistilBERT chosen over Logistic Regression?**

It achieved the strongest verified test performance and captures contextual relationships that sparse TF-IDF features cannot model.

### **42. Why keep Logistic Regression if DistilBERT is better?**

It is a strong, fast, smaller, and more interpretable baseline. It also quantifies the gain from the transformer.

### **43. Why is Naive Bayes useful?**

It is a simple and fast text-classification baseline that provides another comparison point.

### **44. What does TF-IDF do?**

It gives higher weight to terms that are frequent in a ticket but relatively rare across the dataset.

#### **45. Why use unigrams and bigrams?**

Unigrams capture individual words; bigrams capture meaningful phrases such as "admin rights" or "shared mailbox."

#### **46. Why use `min_df=2`?**

It removes features appearing in only one document, reducing noise.

#### **47. Why use `max_df=0.95`?**

It removes terms appearing in more than 95% of documents because they are unlikely to distinguish classes.

#### **48. Why use balanced class weights in Logistic Regression?**

It increases the influence of minority classes during optimization.

#### **49. Why does Naive Bayes have high precision but weak recall?**

It can be conservative for minority classes, predicting them only when evidence is strong, so predictions may be correct but many true cases are missed.

#### **50. What is a confusion matrix?**

It shows actual classes versus predicted classes, making specific confusion patterns visible.

#### **51. What is the best-performing DistilBERT class by F1?**

Purchase, with F1 of 0.9383.

#### **52. What is the weakest DistilBERT class by F1?**

Administrative rights, with F1 of 0.8324.

#### **53. Why might Administrative rights be difficult?**

It is the smallest class and its vocabulary can overlap with Hardware and Access tickets.

#### **54. Why does DistilBERT use max length 128?**

It balances context coverage and computational cost. EDA shows most tickets are shorter, although very long tickets can be truncated.

#### **55. What does truncation mean?**

Tokens beyond the maximum sequence length are removed before model inference.

#### **56. What is mixed precision?**

It uses lower-precision floating point operations where safe to reduce GPU memory and improve training speed.

## **57. What is early stopping?**

It stops training when validation performance stops improving for the configured patience. In the recorded run, all three requested epochs completed.

## **58. What is checkpointing?**

It saves model and training state during training so the best model can be restored and training can potentially resume.

## **59. Why use validation macro F1 as best-model metric?**

It treats every class equally and is more informative than accuracy under imbalance.

## **60. What does `torch.no_grad()` do?**

It disables gradient tracking during inference, reducing memory and computation.

## **61. Why call `model.eval()`?**

It switches layers such as dropout into inference behavior.

## **62. What are logits?**

Logits are raw unnormalized class scores produced by the classification head.

## **63. What does softmax do?**

It converts logits into probabilities that sum to one.

## **64. How is the confidence selected?**

The maximum softmax probability is selected with its label ID.

## **65. What does label mapping do?**

It converts between human-readable categories and integer model labels.

## **66. What is self-attention?**

It lets each token calculate how much information to use from other tokens in the sequence.

## **67. What are query, key, and value vectors?**

They are learned representations used to calculate attention weights and combine token information.

## **68. Why use a rule-based NER system?**

No labeled NER training dataset exists, so configurable rules are honest, explainable, and practical.

## **69. How does case-insensitive entity matching work?**

The EntityRuler uses `phrase_matcher_attr="LOWER"`.

## **70. Why are error-code rules strict?**

To avoid labeling generic phrases such as "an error occurred" as machine-readable error codes.

## **71. Why prefer longer entity phrases?**

access card and conference room are more meaningful than the shorter overlapping words card and room.

## **72. Why is priority independent from classification?**

Category and urgency are separate dimensions. A Hardware ticket can be Critical or Medium.

## **73. How does first-match priority logic work?**

The engine checks Critical before High, Medium, and Low and returns immediately on the first matching canonical rule or alias.

## **74. Why does the engine return a canonical matched rule?**

It normalizes multiple user phrasings into one explainable business rule, such as mapping "unable to log in" to cannot login.

## **75. Why is routing deterministic?**

Organizational ownership is a business rule. Deterministic mapping is predictable, testable, and changeable without retraining.

## **76. Why is routing case-sensitive?**

The current lookup is exact after trimming. Case normalization was not implemented for categories because model labels are controlled.

## **77. Why use frozen dataclasses for results?**

They create small immutable validated value objects for predictable business-logic output.

## **78. Why use FastAPI dependency injection?**

It separates endpoint code from service construction and enables test overrides.

## **79. Why cache the analysis service?**

Loading DistilBERT on every request would be extremely expensive.

## **80. Why use Pydantic response models?**

They document and validate API contracts.

## **81. Why does /analyze persist but /predict does not?**

/analyze represents a completed operational workflow; /predict is a narrower inference endpoint.

## **82. Why store entities as JSON text?**

It keeps the schema simple for variable entity groups. The tradeoff is weaker SQL-level querying.

## **83. Why use UTC timestamps?**

UTC avoids ambiguity across time zones and is a standard persistence practice.

## **84. How are analytics generated?**

SQLAlchemy queries count all rows and group rows by predicted category or priority.

## **85. Why clamp history limit?**

It prevents invalid zero limits and protects the service from excessively large history requests.

## **86. Why does the dashboard call multiple endpoints?**

It was designed to demonstrate individual API capabilities and a full workflow. A more efficient future design would rely mainly on `/analyze`.

## **87. Which dashboard charts use real generated artifacts?**

Model comparison and class distribution.

## **88. Which charts use sample data?**

Priority distribution and routing distribution.

## **89. What does Docker Compose depends\_on do here?**

The dashboard waits for the backend service to become healthy.

## **90. Why are models mounted as a volume?**

The backend needs the saved DistilBERT artifacts, and mounting avoids baking large changing model files permanently into each rebuild.

## **18.3 Advanced Questions**

### **91. Is the `/health` endpoint a deep health check?**

No. It returns a fixed healthy status and does not verify model inference or database access.

### **92. What happens if routing fails after classification?**

`UnsupportedCategoryError` is raised, `/analyze` returns HTTP 400, and persistence does not occur because routing happens before saving.

### **93. What happens if persistence fails?**

The analysis request fails because persistence occurs before the response is returned.

#### **94. Is /analyze transactionally atomic across ML and database processing?**

ML operations are not transactions. The database save itself is committed in one SQLAlchemy session, but there is no distributed transaction around the entire workflow.

#### **95. What concurrency limitation does SQLite have?**

SQLite supports many reads but has limited concurrent write behavior compared with server databases such as PostgreSQL.

#### **96. What risk comes from auto-creating tables?**

`create_all` does not manage schema migrations. Existing schemas may drift when models change.

#### **97. Why is `matched_rule` not in the database?**

The current schema stores priority but not the specific rule. This is a limitation and a useful future schema addition.

#### **98. How would you make routing entity-aware?**

Add a second rule layer after category routing, for example routing `Administrative rights` plus `SOFTWARE=Oracle` to a specialized Oracle administration team.

#### **99. How would you train a priority model?**

Collect human-confirmed priority labels from real workflows, define annotation guidelines, split the data, train/evaluate a classifier, and keep critical keyword rules as a safety override.

#### **100. How would you evaluate the NER system?**

Create a labeled test set with entity boundaries and types, then calculate strict entity-level precision, recall, and F1.

#### **101. What is data leakage and how is it avoided?**

Data leakage occurs when information from validation/test data influences training. The split occurs before fitting the TF-IDF vectorizer, which is fitted only on training text.

#### **102. Why must the same vectorizer be used for baseline inference?**

Classifier coefficients correspond to exact vectorizer feature positions. A different vocabulary produces incompatible vectors.

#### **103. Why might random seeds not guarantee perfectly identical GPU results?**

Some GPU operations and library implementations can remain nondeterministic despite seeding and deterministic settings.

#### **104. How would you monitor model drift?**

Store model version, confidence, human-corrected categories, and timestamps; compare distributions and error rates over time.

#### **105. What security issues exist?**

There is no authentication, authorization, rate limiting, secret management, input-size limit, or protection against exposing sensitive ticket history.

#### **106. How would you secure /history?**

Add authentication, role-based authorization, pagination, redaction, audit logging, and possibly tenant-level filtering.

#### **107. How would you improve API reliability?**

Add structured error handling, startup checks, model/database deep health checks, request IDs, timeouts, observability, and retry strategy where appropriate.

#### **108. What is the risk of returning model confidence as certainty?**

Softmax confidence may be poorly calibrated. It should be treated as a model score, not guaranteed probability of correctness.

#### **109. How would you calibrate confidence?**

Evaluate calibration on validation data and apply methods such as temperature scaling.

#### **110. What happens with an out-of-domain ticket?**

The classifier still selects one of eight classes because there is no unknown class or rejection threshold.

#### **111. How would you add an unknown/review outcome?**

Use confidence and out-of-distribution rules to send uncertain cases to manual review instead of forcing automatic routing.

#### **112. Why can macro precision be high while macro recall is low?**

A model may make few but accurate predictions for minority classes, missing many true cases.

#### **113. What does the Naive Bayes Administrative-rights result show?**

It has precision 1.0 but recall 0.2159, meaning its rare Administrative-rights predictions were correct, but it missed most true cases.

#### **114. Why might Hardware dominate misclassifications?**

It is the largest and broadest category, and many tickets mention hardware terms even when the operational owner is another team.

### **115. Why is DistilBERT not automatically production-ready?**

Good test metrics do not prove latency, scalability, security, monitoring, drift handling, or operational reliability.

### **116. How would you scale inference?**

Use a production model-serving setup, batch requests where useful, add replicas, use GPU or optimized CPU inference, and measure performance.

### **117. How would you reduce DistilBERT inference cost?**

Use quantization, ONNX Runtime, optimized serving, knowledge distillation, shorter max length after analysis, or a baseline fallback.

### **118. Why not combine all business logic into the model?**

Business rules need explainability and frequent changes. Keeping them separate avoids retraining for policy updates.

### **119. How do tests isolate the real transformer?**

API/integration tests use deterministic service stubs, while classifier helper functions and saved artifacts are verified separately.

### **120. What is the strongest honest claim about the project?**

It is a tested end-to-end prototype with real trained baseline and DistilBERT artifacts, deterministic NER/priority/routing logic, persistence, API, dashboard, and container configuration.

---

## **SECTION 19: INTERVIEW PREPARATION**

### **19.1 Architecture and Backend Questions**

#### **Explain the architecture in one minute.**

The project has a modular Python backend. FastAPI exposes endpoints and injects a cached `TicketAnalysisService`. That service loads the saved DistilBERT model once and coordinates entity extraction, priority assignment, routing, and SQLAlchemy persistence. Streamlit calls the API and displays both ticket analysis and report-driven charts. Docker Compose defines separate backend and dashboard services sharing data, models, and reports.

#### **What design pattern does the service layer resemble?**

It resembles a facade/orchestration service: `TicketAnalysisService` provides a unified interface over multiple specialized components. Dependency injection and caching manage its lifetime.

## **Why not instantiate the model inside /predict?**

Loading a transformer is expensive in time and memory. Per-request loading would make latency unacceptable and could exhaust memory.

## **How would you improve separation further?**

Introduce explicit protocols/interfaces for classifier, entity extractor, priority service, router, and repository; inject them into the service constructor; and create environment-specific dependency wiring.

## **What API change would improve dashboard efficiency?**

Use /analyze as the single source for all result panels instead of additionally calling /predict, /entities, and /priority.

## **19.2 ML Questions**

### **Compare Logistic Regression and DistilBERT.**

Logistic Regression learns linear weights over sparse TF-IDF features. It is fast and interpretable but has limited context understanding. DistilBERT uses contextual transformer representations and performs better, but requires more memory and compute.

### **Why does the project need a validation set?**

DistilBERT checkpoint selection and early stopping must not use the final test set. The validation set supports model development while preserving test integrity.

### **How do you know DistilBERT improved performance?**

On the same stratified test split, DistilBERT achieved 0.8878 accuracy and 0.8883 macro F1, compared with Logistic Regression's 0.8616 and 0.8622.

### **What metric would you optimize if minority classes are critical?**

Macro F1, per-class recall, or a business-weighted cost metric. I would inspect Administrative-rights recall specifically.

### **How would you investigate model errors?**

Review the confusion matrix, sample false positives and false negatives by class, inspect ticket length and vocabulary, and identify ambiguous or mislabeled examples.

## **19.3 Database Questions**

### **Why use a repository class?**

It keeps SQLAlchemy queries outside business and API logic, makes persistence testable, and allows later database replacement.

## **Why is JSON text a tradeoff?**

It easily stores variable entity dictionaries, but querying individual entities in SQL is inefficient compared with normalized tables or native JSON types.

## **How would you migrate to PostgreSQL?**

Change the database URL and driver, introduce migrations, verify data types and concurrency behavior, and run repository/integration tests against PostgreSQL.

## **19.4 Testing Questions**

### **Difference between unit and integration tests here?**

Unit tests verify one module's behavior, such as priority or routing. Integration tests combine FastAPI, real rule-based components, and an in-memory database to validate workflow cooperation.

### **Why use an in-memory database?**

It makes tests fast, isolated, and repeatable without modifying the real database.

### **Why do API tests override dependencies?**

They avoid loading the heavy real model and make endpoint responses deterministic.

### **What does 58 passing tests not prove?**

It does not prove production performance, security, real-model end-to-end correctness for every input, or Docker runtime success.

## **19.5 DevOps Questions**

### **Why copy requirements.txt before the project source?**

Docker can cache dependency installation when requirements have not changed, making rebuilds faster.

### **Why does the dashboard use backend:8000 inside Compose?**

Compose service names provide internal DNS. localhost inside the dashboard container would not reach the backend container.

### **Why use a healthcheck?**

It lets Compose determine whether the backend is ready before starting the dependent dashboard workflow.

---

## **SECTION 20: PROJECT DEFENSE**

### **20.1 "Why did you not implement LSTM?"**

Strong answer:

I prioritized a complete, validated production workflow and used Logistic Regression and Naive Bayes as traditional baselines before fine-tuning DistilBERT. The repository contains no LSTM artifact, so I do not claim LSTM results. DistilBERT achieved the strongest verified performance. An LSTM remains a valid future comparison, but implementing it without compromising the API, routing, persistence, dashboard, and testing work was not the chosen scope.

### **20.2 "Why DistilBERT instead of full BERT?"**

DistilBERT offers contextual transformer performance with a smaller model and lower inference cost than full BERT. For an NTCC prototype that must also run as an API and dashboard, it is a practical balance. Full BERT could be evaluated later using the same split.

### **20.3 "Why not use only Logistic Regression?"**

Logistic Regression is strong and efficient, but DistilBERT improved test accuracy from 0.8616 to 0.8878 and macro F1 from 0.8622 to 0.8883. The project keeps Logistic Regression as a meaningful fallback and comparison.

### **20.4 "Why not MongoDB?"**

The persisted data has a stable relational schema and the analytics use counts and grouping. SQLite with SQLAlchemy is simpler for a local prototype. MongoDB could store flexible documents, but it is not necessary for this schema. PostgreSQL would be my preferred production upgrade.

### **20.5 "Why not Flask?"**

FastAPI provides Pydantic validation, type-driven response schemas, dependency injection, and automatic OpenAPI documentation with less manual work. These features fit a typed ML service well.

### **20.6 "Why rule-based NER instead of machine learning?"**

There is no labeled NER training dataset in the repository. Training a model without reliable labels would produce unsupported claims. EntityRuler provides deterministic extraction and can later help bootstrap annotations.

### **20.7 "Your NER has no precision or recall. Is that a weakness?"**

Yes. The implementation is tested behaviorally, but statistical NER evaluation requires

a labeled entity test set. I explicitly mark NER metrics as unverified rather than inventing them.

## **20.8 "Why is priority rule-based?"**

The dataset labels category, not priority. Rules are transparent and auditable. A supervised priority model should only be trained after collecting human-confirmed priority labels.

## **20.9 "Why is default priority Medium?"**

It avoids falsely escalating ordinary tickets to High/Critical and avoids treating unmatched tickets as Low by default. It is a conservative middle state.

## **20.10 "Why does routing ignore entities?"**

The current routing contract maps category to team. Entity-aware routing is a future extension. Keeping the initial mapping deterministic reduces complexity and creates a clear baseline.

## **20.11 "Is the system production-ready?"**

It is a production-oriented prototype with modular code, saved models, validation, persistence, API, dashboard, and deployment configuration. Full production readiness would still require authentication, monitoring, load tests, migrations, security controls, drift detection, and Docker runtime validation.

## **20.12 "Why SQLite for a production-quality project?"**

SQLite is appropriate for the current single-instance prototype and demonstration. SQLAlchemy isolates persistence, so PostgreSQL can replace it for production concurrency and operations.

## **20.13 "How do you handle class imbalance?"**

The project uses stratified splits, reports macro and weighted metrics, and applies balanced class weights in Logistic Regression. DistilBERT does not use weighted sampling in the current implementation.

## **20.14 "Why not use class weights for DistilBERT?"**

It was not implemented in the verified training pipeline. The current model already performs strongly, but class-weighted loss or sampling would be a valid controlled experiment, especially for Administrative rights.

## **20.15 "Why is Administrative rights performance lower?"**

It is the smallest class and overlaps semantically with Access and Hardware. More

examples, error analysis, or class-aware training could improve it.

## 20.16 "Does confidence mean the model is 96% correct?"

No. It is the highest softmax score. Without calibration analysis, it should not be interpreted as a guaranteed probability of correctness.

## 20.17 "What happens if a completely unrelated ticket is submitted?"

The model must still choose one of eight labels because there is no unknown class or rejection threshold. A production system should add uncertainty-based manual review.

## 20.18 "Why do dashboard priority and routing charts use demo data?"

Those charts were implemented before persistence analytics integration. The backend now exposes `/analytics`; connecting the charts to live persisted data is a clear improvement.

## 20.19 "Why are there multiple API calls from the dashboard?"

The dashboard demonstrates each endpoint and the full workflow. For production efficiency, I would render the result from `/analyze` alone.

## 20.20 "How do you prove the project works?"

The repository contains real dataset artifacts, saved baseline and DistilBERT models, generated metrics, module tests, API tests, persistence tests, integration tests, screenshots, and a current result of 58 passing tests.

---

# SECTION 21: FUTURE IMPROVEMENTS

## 21.1 Short-Term Improvements

These are achievable without changing the overall architecture:

1. Use only `/analyze` in the dashboard to reduce repeated inference and persistence side effects. Currently `/analyze` persists once and additional endpoint calls repeat model/rule work.
2. Connect dashboard priority and routing charts to `/analytics`.
3. Store `matched_rule` in the database.
4. Add pagination and filtering to `/history`.

5. Add a startup/deep-health endpoint that verifies model and database readiness.
6. Add request-size limits and clearer exception conversion.
7. Add confidence threshold warnings.
8. Add model version to persisted records.
9. Add more NER aliases and test cases.
10. Add category normalization or explicit case-insensitive route handling if external clients submit categories.

## **21.2 Medium-Term Improvements**

1. Build a labeled NER dataset and measure strict entity precision, recall, and F1.
2. Collect human-confirmed priority labels and evaluate an ML priority model.
3. Add user feedback and corrected categories.
4. Add entity- and priority-aware routing.
5. Add authentication and role-based access.
6. Migrate from SQLite to PostgreSQL.
7. Introduce Alembic migrations.
8. Add model-confidence calibration.
9. Add out-of-distribution/manual-review behavior.
10. Benchmark LSTM honestly as an additional comparison if required.
11. Run repeated training experiments and report mean/std deviation.
12. Add API load and latency tests.

## **21.3 Production-Scale Improvements**

1. Deploy optimized model serving separately from the API orchestration layer.
2. Use container orchestration and horizontal scaling.
3. Add centralized logs, metrics, traces, and alerting.
4. Add queue-based asynchronous processing for large workloads.
5. Add model registry, versioning, and rollback.
6. Monitor data drift, label drift, confidence, and correction rates.
7. Protect sensitive ticket data using encryption, retention policy, and redaction.
8. Integrate with ServiceNow, Jira Service Management, or Freshservice.
9. Add human-in-the-loop review for low-confidence/high-risk cases.
10. Establish retraining and approval pipelines.

## 21.4 Recommended Evaluation Work

- NER evaluation: **UNVERIFIED - REQUIRES REPOSITORY VALIDATION**
  - API latency/throughput: **UNVERIFIED - REQUIRES REPOSITORY VALIDATION**
  - Docker runtime: **UNVERIFIED - REQUIRES REPOSITORY VALIDATION**
  - Model calibration: **UNVERIFIED - REQUIRES REPOSITORY VALIDATION**
  - Repeated-run stability: **UNVERIFIED - REQUIRES REPOSITORY VALIDATION**
- 

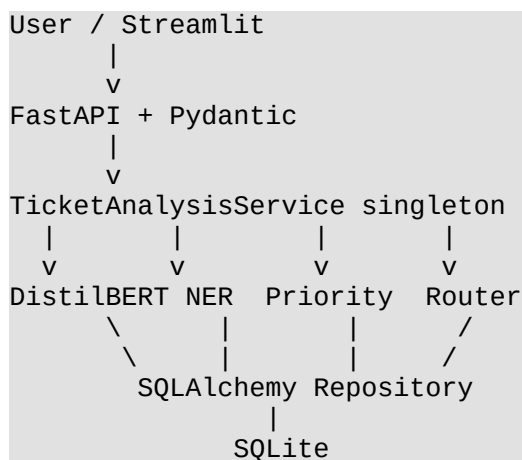
## SECTION 22: PROJECT CHEAT SHEET

Use this section for a 10-minute pre-viva revision.

### 22.1 Project in One Sentence

An end-to-end NLP prototype that classifies IT support tickets with DistilBERT, extracts configured entities, assigns rule-based priority, routes categories to teams, persists analyses in SQLite, and exposes results through FastAPI and Streamlit.

### 22.2 Architecture



### 22.3 Dataset

| Fact    | Value                                              |
|---------|----------------------------------------------------|
| File    | data/<br>all_tickets_processed_improved_v<br>3.csv |
| Rows    | 47,837                                             |
| Columns | Document, Topic_group                              |
| Classes | 8                                                  |
| Split   | Stratified 80/10/10                                |

| Fact                 | Value                  |
|----------------------|------------------------|
| Train / Val / Test   | 38,269 / 4,784 / 4,784 |
| Average words        | 43.60                  |
| Median words         | 26                     |
| Imbalance ratio      | 7.7369                 |
| Missing / duplicates | 0 / 0                  |

## 22.4 Classes and Routes

| Category              | Team                       |
|-----------------------|----------------------------|
| Hardware              | Hardware Team              |
| HR Support            | HR Team                    |
| Access                | Access Management Team     |
| Storage               | Storage Team               |
| Purchase              | Procurement Team           |
| Administrative rights | System Administration Team |
| Internal Project      | Internal Projects Team     |
| Miscellaneous         | Service Desk Team          |

## 22.5 Models and Metrics

| Model               | Accuracy | Macro Precision | Macro Recall | Macro F1 |
|---------------------|----------|-----------------|--------------|----------|
| Naive Bayes         | 0.7795   | 0.8802          | 0.6814       | 0.7336   |
| Logistic Regression | 0.8616   | 0.8507          | 0.8762       | 0.8622   |
| DistilBERT          | 0.8878   | 0.8922          | 0.8847       | 0.8883   |

DistilBERT is final. LSTM is not implemented.

## 22.6 DistilBERT Configuration

- distilbert-base-uncased
- 8 labels
- max length 128
- batch size 16
- 3 epochs
- learning rate 2e - 5

- weight decay 0.01
- seed 42
- mixed precision in recorded CUDA run
- best model selected by validation macro F1
- best checkpoint checkpoint -7176

## 22.7 Preprocessing

```
NFKC Unicode normalize
-> lowercase
-> remove URLs
-> remove emails
-> normalize control characters
-> filter unsupported characters while preserving technical tokens
-> collapse whitespace
```

No stemming, lemmatization, or stop-word removal.

## 22.8 Entity Types

- SOFTWARE
- DEVICE
- ERROR\_CODE
- SYSTEM
- LOCATION

Implementation: `spacy.blank("en") + EntityRuler + config/entity_patterns.json`.

## 22.9 Priority Rules

```
Critical: server down, production down, outage, system unavailable, data loss
High: cannot login, access denied, vpn not working, email not working,
application unavailable
Medium: installation, software request, upgrade, configuration
Low: information request, documentation, inquiry
Default: Medium, matched_rule = null
```

Highest-priority first match wins. Aliases return canonical rules.

## 22.10 API Endpoints

| Method | Endpoint | Purpose                     |
|--------|----------|-----------------------------|
| GET    | /health  | Fixed health status         |
| GET    | /metrics | Runtime model/rule metadata |
| GET    | /history | Recent persisted analyses   |

| Method | Endpoint   | Purpose                               |
|--------|------------|---------------------------------------|
| GET    | /analytics | Count/category/priority distributions |
| POST   | /predict   | Classification                        |
| POST   | /entities  | Entity extraction                     |
| POST   | /priority  | Priority assignment                   |
| POST   | /route     | Category routing                      |
| POST   | /analyze   | Full workflow plus persistence        |

Swagger: /docs.

## 22.11 Database

Table: ticket\_predictions

```
id
ticket_text
predicted_category
confidence_score
entities_json
priority
assigned_team
created_at
```

SQLite file: data/ticket\_routing.db.

## 22.12 Dashboard

- Sidebar API URL and health.
- Analyze Ticket tab.
- Prediction, entities, priority, routing, full-analysis panels.
- Analytics tab.
- Real model comparison and class-distribution charts.
- Demo priority and routing charts.

## 22.13 Docker

```
backend: Uvicorn, port 8000
dashboard: Streamlit, port 8501
BACKEND_URL=http://backend:8000
volumes: data, models, reports
```

Command:

```
docker compose up --build
```

Docker runtime success: **UNVERIFIED - REQUIRES REPOSITORY VALIDATION.**

## 22.14 Testing

58 passed in 16.41s

Coverage includes preprocessing, baselines, DistilBERT helpers, NER, priority, routing, API, database, dashboard utilities, Docker config, and integration.

## 22.15 Five Critical Defense Statements

1. The repository has eight classes and 47,837 tickets.
2. DistilBERT's verified test accuracy is 0.8878 and macro F1 is 0.8883.
3. No LSTM model exists; never claim LSTM metrics.
4. NER is rule-based EntityRuler; never claim trained-NER metrics.
5. The API is FastAPI, not Flask.

## 22.16 Demo Script

1. Start API: `uvicorn src.api.app:app --reload`.
2. Open `/docs` and call `/health`.
3. Call `/analyze` with:

I am unable to log in to Outlook on my laptop.

1. Explain category/confidence, SOFTWARE and DEVICE entities, High priority, canonical cannot login matched rule, routing team, and automatic persistence.
2. Show `/history` and `/analytics`.
3. Open Streamlit dashboard.
4. Show Analytics model comparison.
5. State limitations honestly.

## 22.17 Final Memory Hook

47,837 tickets  
8 classes  
3 verified models  
DistilBERT: 88.78% accuracy, 0.8883 macro F1  
5 entity types  
4 priorities  
8 routing mappings  
9 project API endpoints  
1 SQLite table  
58 passing tests